

Algorytmy Zaawansowane

Opracował Radosław Głasek na podstawie wykładów Michała Tuczyńskiego

Spis treści

1	Wykład 1	2
1.1	Notacja asymptotyczna — przypomnienie	2
1.2	Strategia typu <i>Dziel i Zwyciężaj</i> (<i>Divide and Conquer</i>)	3
2	Wykład 2	6
2.1	Algorytm Strasiery	6
2.2	Własność optymalnej podstruktury	7
2.3	Znajdywanie pary najbliższych punktów	7
3	Wykład 3	9
3.1	Programowanie dynamiczne	9
3.2	Algorytm mnożenia macierzy	10
3.3	Problem mnożenia ciągu macierzy	10
3.4	Problem najdłuższego wspólnego podciągu	15

Wykład 1

Notacja asymptotyczna — przypomnienie

$$f, g : \mathbb{N} \rightarrow \mathbb{R}_+$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} g(n) \leq c \cdot f(n)$$

$$g(n) = \Omega(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c \cdot f(n) \leq g(n)$$

$$g(n) = \Theta(f(n)) \Leftrightarrow \exists_{c_1, c_2 \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$$

$$g(n) = \Theta(f(n)) \Leftrightarrow (g(n) = \mathcal{O}(f(n)) \wedge g(n) = \Omega(f(n)))$$

$$n^a = \mathcal{O}(n^b), \quad 0 < a \leq b$$

$$\log n = \mathcal{O}(n^\epsilon), \quad \epsilon > 0$$

$$n^a = \mathcal{O}(b^n), \quad 1 < a \leq b$$

$$\log_a n = \Theta(\log_b n), \quad a, b > 1$$

$$n^a = \mathcal{O}(b^n), \quad 0 < a, \quad b > 1$$

$$T(n) = 5n^3 + 2n^2 + 100n \log n \underset{n \rightarrow \infty}{\approx} 5n^3 = \Theta(n^3)$$

Procedura rekurencyjna

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n \geq 1 \end{cases}$$

Oczywiste jest, że $n! = \prod_{i=1}^n i$

Ciąg Fibonacciego F_n - ciąg Fibonacciego.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n \geq 2 \end{cases}$$

Algorytm 1: Rekurencyjna konstrukcja ciągu Fibonacciego

```

function FIBB( $n$ )
1:   if  $n = 0$  then return 0
2:   if  $n = 1$  then return 1
3:   return FIBB( $n - 1$ ) + FIBB( $n - 2$ )
    
```

Wywołanie rekurencyjne dla (typowo łatwiejszych i mniejszych) instancji tego samego problemu.

Strategia typu *Dziel i Zwyciężaj* (*Divide and Conquer*)

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze podproblemy
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów

Zakładamy, że (zazwyczaj) w fazie *dziel* problem jest dzielony na co najmniej dwa podproblemy i podproblemy są rozłączne.

Przykład 1.1. Mnożenie liczb n -cyfrowych x, y – liczby naturalne n -cyfrowe.

Aby policzyć $x \cdot y$ można użyć mnożenia pisemnego (jak na kartce). Tym sposobem musimy wykonać $\mathcal{O}(n^2)$ mnożeń cyfr. Spróbujemy znaleźć lepszą metodę. Dzielimy x i y na „dwie połowy”. Zakładamy, dla uproszczenia, że n jest parzyste.

$$x = x_1 x_2 \cdots x_n = \underbrace{x_1 x_2 \cdots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \cdots x_n}_{x_R} = 10^{n/2} x_L + x_R$$

podobnie dla y :

$$y = \cdots = 10^{n/2} y_L + y_R$$

Wymnażamy:

$$x \cdot y = (10^{n/2} x_L + x_R) \cdot (10^{n/2} y_L + y_R) = 10^n x_L y_L + 10^{n/2} (x_L y_R + x_R y_L) + x_R y_R$$

Złożoność czasowa algorytmu – ozn. $T(n)$.

- Podział x, y na x_L, x_R, y_L, y_R zajmie $\mathcal{O}(n)$ czasu.
- Dodanie liczb n -cyfrowych zajmie $\mathcal{O}(n)$ czasu.
- Mnożenie przez 10^n jest równoważne dopisaniu n zer więc też $\mathcal{O}(n)$.

$$T(n) = 4 \cdot T(n/2) + \Theta(n)$$

Można lepiej, bo możemy zauważyć, że $x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$, więc potrzebujemy policzyć tylko 3 iloczyny: $x_L y_L, x_R y_R, (x_L - x_R) \cdot (y_R - y_L)$. To daje $T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$

Twierdzenie 1.2. *CLRS - Uniwersalne Twierdzenie o Rekurencji* Niech $a \geq 1, b > 1$ będą stałymi, $f(n)$ funkcją i $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \begin{array}{l} f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ a \cdot f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{array} \end{cases}$$

Lemat 1.3. Niech $a \geq 1, b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją zdefiniowaną dla potęg b . Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta(n^{\log_b n}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b a \cdot \log_b n} = (b^{\log_b a})^{\log_b n} = n^{\log_b a} \quad (1)$$

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a \left(f\left(\frac{n}{b}\right) + aT\left(\frac{n}{b^2}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2T\left(\frac{n}{b^2}\right) = f(n) + af\left(\frac{n}{b}\right) + a^2 \left(f\left(\frac{n}{b^2}\right) + aT\left(\frac{n}{b^3}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n}}\right) + a^{\log_b n} T(1) \\ &\stackrel{1}{=} \dots + a^{\log_b n} T(1) = \dots + n^{\log_b a} T(1) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

□

Przeprowadzimy teraz dowód uniwersalnego twierdzenia o rekurencji (1.2). Ograniczymy się do przypadku, gdy n jest potęgą b . Pozostały przypadek jest bardziej techniczny.

Dowód. Rozważmy wszystkie przypadki:

1. $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$. Wtedy

$$f\left(\frac{n}{b^j}\right) = \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right)$$

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) = \mathcal{O}\left(\underbrace{\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}}_{\blacktriangle}\right) \\ \blacktriangle &= \sum_{j=0}^{\log_b n - 1} a^j \frac{n^{\log_b a - \epsilon}}{b^{j(\log_b a - \epsilon)}} = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} a^j \frac{b^{j\epsilon}}{b^{j \log_b a}} \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\epsilon}{\underbrace{b^{\log_b a}}_{=a}}\right)^j = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} b^{\epsilon j} \quad (\text{wzór na ciąg geometryczny}) \\ &= n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \frac{n^\epsilon - 1}{\underbrace{b^\epsilon - 1}_{\text{stała}}} \\ &= n^{\log_b a - \epsilon} \mathcal{O}(n^\epsilon - 1) = \mathcal{O}(n^{\log_b a - \epsilon} \cdot n^\epsilon - 1) = \mathcal{O}(n^{\log_b a}) \end{aligned}$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n - 1} a^j T\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a})$$

2. $f(n) = \Theta(n^{\log_b a})$. Wtedy:

$$\begin{aligned}
 f\left(\frac{n}{b^j}\right) &= \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n-1} a^j \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{b^{j \log_b a}}\right) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{\underbrace{b^{\log_b a^j}}_{=a^j}}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1\right) = \Theta(n^{\log_b a} \log_b n) \\
 \text{Z lematu: } T(n) &= \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a} \log n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log n)
 \end{aligned}$$

3. $f(n) = \Omega(n^{\log_b a + \epsilon})$ oraz $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ dla jakiegoś $c < 1$ i odpowiednio dużych n .

Z założenia $f\left(\frac{n}{b}\right) \leq \frac{c}{a} f(n)$, więc $f\left(\frac{n}{b^j}\right) \leq \left(\frac{c}{a}\right)^j f(n) = \frac{c^j}{a^j} f(n)$.

$$\begin{aligned}
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &\leq \sum_{j=0}^{\log_b n-1} \cancel{a^j} \frac{c^j}{\cancel{a^j}} f(n) + \mathcal{O}(1) \leq f(n) \sum_{j=0}^{\log_b n-1} c^j + \mathcal{O}(1) \\
 &\leq f(n) \sum_{j=0}^{\infty} c^j + \mathcal{O}(1) \leq f(n) \frac{1}{1-c} + \mathcal{O}(1) = \mathcal{O}(f(n))
 \end{aligned}$$

$$\text{Z drugiej strony: } \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) = a^0 f\left(\frac{n}{b^0}\right) + \underbrace{\sum_{j=1}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right)}_{\geq 0} = \Omega(f(n))$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \Theta(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

□

Wniosek 1.4. Niech $a \geq 1$, $b > 1$ będą stałymi, a $f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$. Niech $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a > b^k \Leftrightarrow \log_b a > k \\ \Theta(n^{\log_b a} \log n) & a = b^k \Leftrightarrow \log_b a = k \\ \Theta(n^k) & a < b^k \Leftrightarrow \log_b a < k \end{cases}$$

Wróćmy teraz do naszego problemu mnożenia liczb n -cyfrowych. Drugi algorytm ma złożoność:

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 4, b = 2, k = 1$$

$$4 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Trzeci algorytm ma złożoność:

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 3, b = 2, k = 1$$

$$3 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.59})$$

Wykład 2

Przykład 2.1. Mnożenie macierzy

A, B - macierze $n \times n$

$$A \cdot B = C = ?$$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $\mathcal{O}(n^3)$. Musimy obliczyć n^2 wartości i obliczenie każdej zajmuje liniowy czas.

$$B = \begin{bmatrix} \vdots \end{bmatrix} \quad A = \begin{bmatrix} \dots \end{bmatrix} \begin{bmatrix} \times \end{bmatrix} = C$$

Algorytm Strassera

Zakładamy, że n jest potęgą 2 (dla przejrzystości). Dzielimy A i B na 4 macierze $\frac{n}{2} \times \frac{n}{2}$:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right], \quad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

$$A \cdot B = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \cdot \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right] = \left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right]$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Zamieniliśmy 1 mnożenie macierzy $n \times n$ na 8 mnożeń macierzy $\frac{n}{2} \times \frac{n}{2}$ + podział i dodawanie ($\mathcal{O}(n^2)$). Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2) \quad a = 8, b = 2, k = 2$$

$$8 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy

$$M_1 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

$$\text{to } C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$$a = 7, b = 2, k = 2$$

$$7 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

Własność optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie można skonstruować z optymalnych rozwiązań podproblemów). Żeby metoda *dziel i zwyciężaj* działała dobrze muszą być spełnione dwa warunki (przez problem i algorytm):

1. własność optymalnej podstruktury
2. podproblemy są rozłączne

Znajdywanie pary najbliższych punktów

Q - zbiór $n \geq 2$ punktów na płaszczyźnie

$$p_1 = (x_1, y_1) \quad p_2 = (x_2, y_2) \quad d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Problem: znaleźć parę punktów w najmniejszej odległości. Algorytm siłowy (*brute force*), który sprawdza wszystkie pary punktów działa w czasie $\Theta(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ par punktów. Pokażemy algorytm typu *dziel i zwyciężaj*, który działa w czasie $\mathcal{O}(n \log n)$.

W każdym wywołaniu rekurencyjnym wejście składa się z 3 rzeczy: zbioru $P \subset Q$ i dwóch tablic X i Y które zawierają wszystkie punkty z P . W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y po drugiej współrzędnej.

Opis algorytmu:

jeśli $|P| \leq 3$ sprawdzamy wszystkie pary

jeśli $|P| > 3$

Dziel: Dzielimy punkty P pionową prostą l na dwa podzbiory P_L i P_R na lewo i na prawo od l tak, że $|P_L| = \left\lceil \frac{|P|}{2} \right\rceil$, $|P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$. Dzielimy tablicę X na X_L i X_R zawierające punkty odpowiednio P_L i P_R posortowane w porządku niemalejącym względem pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające odpowiednio punkty z P_L i P_R posortowane w porządku niemalejącym po drugiej współrzędnej.

Zwyciężaj: Wywołujemy rekurencyjnie algorytm dla wejścia P_L, X_L, Y_L i P_R, X_R, Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla odpowiednio P_L i P_R . Niech δ będzie mniejszą z nich: $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów z P składa się z dwóch punktów z P_L lub z dwóch punktów z P_R lub jednego punktu z P_L i jednego z P_R .

Musimy sprawdzić czy istnieje para 3-go typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia czy taka para istnieje wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.

2. Dla każdego punktu p z Y' sprawdzamy odległość między p , a p' dla 7 kolejnych punktów p' z Y' . Algorytm oblicza najmniejszą odległość δ' i zapamiętuje ją i odpowiednią parę punktów.
3. Jeśli $\delta' < \delta$ to pionowy pas zawiera parę punktów w mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. Wpp. zwraca δ i odpowiednią parę punktów.

Dowód poprawności. Jedyną rzeczą, która nie jest oczywista jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie łącz. Przypuśćmy, że p, q są parą punktów w najmniejszej odległości w P i $\delta(p, q) = \delta'$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$. Bez straty ogólności załóżmy $y_p \leq y_q$. Pokażemy, że algorytm znajdzie tę parę punktów. Przypuśćmy, że nie. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$ to $y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \delta(p, q) \leq \delta$. Ponadto y_r musi być między y_p i y_q : $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$ zatem co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$, $y = y_p + \delta$.

Wtedy w lewym lub prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat. Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją dwa punkty w P_L w odległości mniejszej niż δ , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów dla których odległość między dowolnymi dwoma jest mniejsza niż δ . Sprzeczność (bo najmniejsza odległość w P_L to $\delta_L \geq \delta$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

Analiza złożoności czasowej Na początku musimy posortować tablicę X i Y . To zajmuje $\mathcal{O}(n \log n)$. Jeśli X i Y są już posortowane to podział X na posortowane X_L i X_R oraz Y analogicznie zajmie $\mathcal{O}(n)$.

Algorytm 2: ???

```

1: length( $Y_L$ )  $\leftarrow$  length( $Y_R$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $Y(i) \in P_L$  then
4:     length( $Y_L$ )  $\leftarrow$  length( $Y_L$ ) + 1
5:      $Y_L(\text{length}(Y_L)) \leftarrow Y(i)$ 
6:   else
7:     length( $Y_R$ )  $\leftarrow$  length( $Y_R$ ) + 1
8:      $Y_R(\text{length}(Y_R)) \leftarrow Y(i)$ 

```

Mamy dwa wywołania rekurencyjne dla $n/2$ punktów i fazę łącz. Tablicę Y' można otrzymać z Y w czasie $\mathcal{O}(n)$ tak, że Y' jest posortowana. Niech $x = x_L$ będzie równanie prostej l .

Algorytm 3: ???

```

1: length( $Y'$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $x_L - \delta \leq Y(i).x \leq x_L + \delta$  then
4:     length( $Y'$ )  $\leftarrow$  length( $Y'$ ) + 1
5:      $Y'(\text{length}(Y')) \leftarrow Y(i)$ 

```

Skoro dla każdego z $\mathcal{O}(n)$ punktów z Y' sprawdzamy odległość tylko co najwyżej 7 punktów, to najmniejsza odległość δ' punktów z Y' znajdziemy w czasie $\mathcal{O}(n)$.

Oznaczmy złożoność algorytmu bez wstępnego sortowania przez $T(n)$. Mamy:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \quad b = 2, \quad k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n \log n)$$

Wykład 3

Programowanie dynamiczne

Czasami gdy zaimplementujemy jakąś formę rekurencyjną dostajemy algorytm działający w czasie wykładniczym. Powodem może być to, że jakieś wartości obliczane są wielokrotnie.

Problem liczb Fibonacciego

$$\begin{cases} F_0 = F_1 = 1 & n = 0, 1 \\ F_n = F_{n-1} + F_{n-2} & n \geq 2 \end{cases} \qquad \text{Zadanie: Obliczyć } n\text{-tą liczbę Fibonacciego}$$

Algorytm 4: Algorytm naiwny Fib1

```
function FIB1(n)
1:   if n ≤ 1 then return 1
2:   return FIB1(n - 1) + FIB1(n - 2)
```

Zobaczmy jak jest obliczane np. $\text{FIB1}(5)$. W drzewie wywołań poszczególne wartości powtarzają się:

$F(i)$ jest obliczane	razy
(1)	5
(2)	3
(3)	2
(4)	1
(5)	1

Niech $s(n)$ będzie liczbą dodawań wykonanych przez alg. w wywołaniu $\text{FIB1}(n)$.

$$\begin{cases} s(0) = s(1) = 0 \\ s(n) = s(n-1) + s(n-2) + 1 & n \geq 2 \end{cases}$$

Niech $f(n) = s(n) + 1$

Wtedy $f(0) = f(1) = 1$

$$\begin{aligned} f(n) &= s(n) + 1 = s(n-1) + s(n-2) + 1 + 1 = f(n-1) - 1 + f(n-2) - 1 + 1 + 1 \\ &= f(n-1) + f(n-2) \end{aligned}$$

Zatem $f(n) = F_n$ $f(n) = \Theta(F_n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) = \Omega(1.61^n)$. Zatem $s(n) = \Omega(1.61^n)$.

FIB1 jest algorytmem wykładniczym.

Oto alg. progr. dynamicznego:

Algorytm 5: Algorytm dynamiczny Fib2

<pre> function FIB2(<i>n</i>) 1: if <i>n</i> ≤ 1 then return 1 2: <i>F</i>(0) ← <i>F</i>(1) ← 1 3: for <i>i</i> ← 2 to <i>n</i> do 4: <i>F</i>(<i>i</i>) ← <i>F</i>(<i>i</i> − 1) + <i>F</i>(<i>i</i> − 2) 5: return <i>F</i>(<i>n</i>) </pre>	▷ <i>F</i> - tablica
---	-----------------------------

Liczba dodawań wykonanych przez FIB2 w wywołaniu FIB2(*n*) wynosi $n - 1$. Złożoność czasowa wynosi $\Theta(n)$.

Algorytm mnożenia macierzy

Algorytm 6: Naiwne mnożenie macierzy

<pre> function MATRIXMULTIPLY(<i>A</i>, <i>B</i>) 1: for <i>i</i> ← 1 to <i>p</i> do 2: for <i>j</i> ← 1 to <i>r</i> do 3: <i>C</i>(<i>i</i>, <i>j</i>) ← 0 4: for <i>k</i> ← 1 to <i>q</i> do 5: <i>C</i>(<i>i</i>, <i>j</i>) ← <i>C</i>(<i>i</i>, <i>j</i>) + <i>A</i>(<i>i</i>, <i>k</i>) · <i>B</i>(<i>k</i>, <i>j</i>) 6: return <i>C</i> </pre>	▷ <i>A</i> – macierz $p \times q$; <i>B</i> – macierz $q \times r$; <i>C</i> – macierz $p \times r$ ▷ operacja dominująca
--	---

Operacją dominującą jest mnożenie w linii 5. Mamy $p \cdot q \cdot r$ mnożeń. Złożoność jest $\mathcal{O}(p \cdot q \cdot r)$.

Problem mnożenia ciągu macierzy

Przykład: (A_1, A_2, A_3) gdzie A_1 – macierz 5×10 , A_2 – macierz 10×3 , A_3 – macierz 3×2 . Ile potrzeba mnożeń skalarnych (mnożenia liczb) do obliczenia iloczynu $A_1 \cdot A_2 \cdot A_3$?

$$A_1 \cdot A_2 \cdot A_3 = (A_1 \cdot A_2) \cdot A_3 = A_1 \cdot (A_2 \cdot A_3)$$

I sposób: $(A_1 \cdot A_2) \cdot A_3 = 5 \cdot 10 \cdot 3 + 5 \cdot 3 \cdot 2 = 150 + 30 = 180$

II sposób: $A_1 \cdot (A_2 \cdot A_3) = 10 \cdot 3 \cdot 2 + 5 \cdot 10 \cdot 2 = 60 + 100 = 160$

II sposób jest szybszy.

Problem: Dla danego ciągu macierzy $(A_1, A_2, \dots, A_n)$, gdzie A_i jest macierzą $p_{i-1} \times p_i$, znajdź kolejność wykonywania mnożeń minimalizującą liczbę mnożeń skalarnych użytych do obliczenia iloczynu $A_1 \cdot A_2 \cdot \dots \cdot A_n$.

Ile jest różnych sposobów obliczenia tego iloczynu? Innymi słowy, na ile sposobów możemy poprawnie wstawić nawiasy w iloczynie $A_1 \cdot A_2 \cdot \dots \cdot A_n$?

Przykład: $A_1 \cdot A_2 \cdot A_3 \cdot A_4$ ($n = 4$)

- $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$
- $(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$

- $A_1 \cdot ((A_2 \cdot A_3) \cdot A_4)$
- $A_1 \cdot (A_2 \cdot (A_3 \cdot A_4))$
- $(A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$

Zatem mamy 5 sposobów.

Sprawdźmy czy umiemy rozwiązać ten problem szybko poprzez sprawdzenie wszystkich możliwości. Niech $P(n)$ – liczba sposobów wstawienia nawiasów w iloczynie n macierzy. Załóżmy, że ostatnie mnożenie w ciągu $A_1 \dots A_n$ to mnożenie między A_k i A_{k+1} : $(A_1 \dots A_k)(A_{k+1} \dots A_n)$. W podciągach lewym i prawym nawiasy są rozmieszczane niezależnie od siebie.

Stąd rekurencja:

$$\begin{cases} P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \\ P(1) = 1 \end{cases}$$

Jest to znana formuła rekurencyjna a rozwiązanie to przesunięta liczba Catalana:

$$P(n) = C(n-1), \text{ gdzie } C(n) = \frac{1}{n+1} \binom{2n}{n} \text{ jest } n\text{-tą liczbą Catalana.}$$

Wiadomo, że $C(n) = \Omega\left(\frac{4^n}{n^{3/2}}\right)$. Zatem $P(n)$ jest wykładniczy względem n . Chcemy znaleźć szybki algorytm.

Rozwiązanie z optymalną podstrukturą: Zauważmy że jeśli optymalne rozwiązanie jest postaci $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ dla jakiegoś $k \in \{1, \dots, n-1\}$, to w podciągach $A_1 \dots A_k$ i $A_{k+1} \dots A_n$ rozwiązanie nawiasów musi być optymalne. Wpp. lepsze rozmieszczenie nawiasów w $A_1 \dots A_k$ lub $A_{k+1} \dots A_n$ dałoby lepsze rozmieszczenie nawiasów w całym ciągu.

Rozwiązanie optymalne zawiera rozwiązania optymalne podproblemów (własność optymalnej podstruktury). Podproblemami w naszym problemie są problemy optymalnego rozmieszczenia nawiasów w ciągach $A_i A_{i+1} \dots A_j$ dla $1 \leq i \leq j \leq n$.

Niech $m(i, j)$ oznacza minimalną liczbę mnożeń skalarnych potrzebnych do obliczenia tego iloczynu $A_i \dots A_j$ dla $1 \leq i \leq j \leq n$. Chcemy obliczyć $m(1, n)$.

$$m(i, i) = 0$$

Optymalny sposób rozmieszczenia nawiasów jest postaci $(A_i \dots A_k)(A_{k+1} \dots A_j)$ dla pewnego $k \in \{i, \dots, j-1\}$. Stąd mamy zależność:

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} \cdot p_k \cdot p_j\} \quad \text{dla } i < j$$

Niech $s(i, j)$ będzie wartością k , dla którego to minimum jest osiągnięte (czyli $s(i, j)$ wskazuje miejsce ostatniego mnożenia w optymalnym rozwiązaniu podproblemu $A_i \dots A_j$). Czyli dla $1 \leq i < j \leq n$.

Jeśli zaimplementujemy po prostu tę formułę rekurencyjną otrzymamy alg. wykładniczy. Ale jest tylko $n + \binom{n}{2} = \mathcal{O}(n^2)$ podproblemów, więc rzędu $\mathcal{O}(n^2)$ liczb $m(i, j)$ i każdą z tych liczb wystarczy obliczyć jeden raz.

Algorytm 7: Algorytm Matrix Chain Order

```

function MATRIXCHAINORDER( $p$ )     $\triangleright (p_0, p_1, \dots, p_n)$  - ciąg rozmiarów;  $A_i$  - macierz  $p_{i-1} \times p_i$ 
1:    $n \leftarrow \text{length}(p) - 1$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:      $m(i, i) \leftarrow 0$ 
4:     for  $l \leftarrow 2$  to  $n$  do                                 $\triangleright l$  - liczba macierzy w podproblemie
5:       for  $i \leftarrow 1$  to  $n - l + 1$  do                         $\triangleright l = 2 : A_1 A_2, A_2 A_3, \dots, A_{n-1} A_n$ 
6:          $j \leftarrow i + l - 1$                                  $\triangleright l = 3 : A_1 A_2 A_3, A_2 A_3 A_4, \dots, A_{n-2} A_{n-1} A_n$ 
7:          $m(i, j) \leftarrow \infty$                                  $\triangleright l = n : A_1 \dots A_n$ 
8:         for  $k \leftarrow i$  to  $j - 1$  do
9:            $q \leftarrow m(i, k) + m(k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
10:        if  $q < m(i, j)$  then
11:           $m(i, j) \leftarrow q$ 
12:           $s(i, j) \leftarrow k$ 
13:   return  $m, s$ 

```

Mamy 3 pętle (po l, i, k) i liczbę iteracji każdej z tych pętli można ograniczyć przez n , zatem złożoność czasowa to $\mathcal{O}(n^3)$.

Przykład: $n = 5$, $p = (4, 5, 2, 1, 2, 3)$.

$A_1 - 4 \times 5$, $A_2 - 5 \times 2$, $A_3 - 2 \times 1$, $A_4 - 1 \times 2$, $A_5 - 2 \times 3$.

$$m \left[\begin{array}{c|c|c|c|c|c} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 40 & 30 & 38 & 48 \\ \hline 2 & \times & 0 & 10 & 20 & 31 \\ \hline 3 & \times & \times & 0 & 4 & 12 \\ \hline 4 & \times & \times & \times & 0 & 6 \\ \hline 5 & \times & \times & \times & \times & 0 \end{array} \right]$$

$$s \left[\begin{array}{c|c|c|c|c|c} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & \times & 1 & 1 & 3 & 3 \\ \hline 2 & \times & \times & 2 & 3 & 3 \\ \hline 3 & \times & \times & \times & 3 & 3 \\ \hline 4 & \times & \times & \times & \times & 4 \\ \hline 5 & \times & \times & \times & \times & \times \end{array} \right]$$

$$\begin{aligned}
 L = 2 : \quad & m(1, 2) = 4 \cdot 5 \cdot 2 = 40 \\
 & m(2, 3) = 5 \cdot 2 \cdot 1 = 10 \\
 & m(3, 4) = 2 \cdot 1 \cdot 2 = 4 \\
 & m(4, 5) = 1 \cdot 2 \cdot 3 = 6
 \end{aligned}$$

$$\begin{aligned}
 L = 3 : \quad & m(1, 3) = \min \begin{cases} m(1, 1) + m(2, 3) + 4 \cdot 5 \cdot 1 \\ m(1, 2) + m(3, 3) + 4 \cdot 2 \cdot 1 \end{cases} = \min \begin{cases} 0 + 10 + 20 \\ 40 + 0 + 8 \end{cases} = \min \begin{cases} 30 \\ 48 \end{cases} = 30 \\
 & m(2, 4) = \min \begin{cases} m(2, 2) + m(3, 4) + 5 \cdot 2 \cdot 2 \\ m(2, 3) + m(4, 4) + 5 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 4 + 20 \\ 10 + 0 + 10 \end{cases} = \min \begin{cases} 24 \\ 20 \end{cases} = 20 \\
 & m(3, 5) = \min \begin{cases} m(3, 3) + m(4, 5) + 2 \cdot 1 \cdot 3 \\ m(3, 4) + m(5, 5) + 2 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 6 + 6 \\ 4 + 0 + 12 \end{cases} = \min \begin{cases} 12 \\ 16 \end{cases} = 12
 \end{aligned}$$

$$\begin{aligned}
 L = 4 : \quad & m(1, 4) = \min \begin{cases} m(1, 1) + m(2, 4) + 4 \cdot 5 \cdot 2 \\ m(1, 2) + m(3, 4) + 4 \cdot 2 \cdot 2 \\ m(1, 3) + m(4, 4) + 4 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 20 + 40 \\ 40 + 4 + 16 \\ 30 + 0 + 8 \end{cases} = \min \begin{cases} 60 \\ 60 \\ 38 \end{cases} = 38 \\
 & m(2, 5) = \min \begin{cases} m(2, 2) + m(3, 5) + 5 \cdot 2 \cdot 3 \\ m(2, 3) + m(4, 5) + 5 \cdot 1 \cdot 3 \\ m(2, 4) + m(5, 5) + 5 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 12 + 30 \\ 10 + 6 + 15 \\ 20 + 0 + 30 \end{cases} = \min \begin{cases} 42 \\ 31 \\ 50 \end{cases} = 31
 \end{aligned}$$

$$L = 5 : \quad m(1, 5) = \min \begin{cases} m(1, 1) + m(2, 5) + 4 \cdot 5 \cdot 3 \\ m(1, 2) + m(3, 5) + 4 \cdot 2 \cdot 5 \\ m(1, 3) + m(4, 5) + 4 \cdot 1 \cdot 3 \\ m(1, 4) + m(5, 5) + 4 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 31 + 60 \\ 40 + 12 + 24 \\ 30 + 6 + 12 \\ 38 + 0 + 24 \end{cases} = \min \begin{cases} 91 \\ 76 \\ 48 \\ 62 \end{cases} = 48$$

Oto algorytm znajdowania optymalnego rozwiązania:

Algorytm 8: Algorytm Matrix Chain Multiply

function MATRIXCHAINMULTIPLY(A, s, i, j)	$\triangleright A = (A_1, \dots, A_n)$
1: if $j = i$ then	
2: return A_i	
3: $X \leftarrow$ MATRIXCHAINMULTIPLY($A, s, i, s(i, j)$)	
4: $Y \leftarrow$ MATRIXCHAINMULTIPLY($A, s, s(i, j) + 1, j$)	
5: return MATRIXMULTIPLY(X, Y)	

W celu znalezienia rozwiązania wywołujemy: MATRIXCHAINMULTIPLY($A, s, 1, n$)

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \cdot A_5$$

$$s(1, 5) = 3 \implies (A_1 \cdot A_2 \cdot A_3)(A_4 \cdot A_5)$$

$$s(1, 3) = 1 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5) \quad s(4, 5) = 4 \implies \text{bez zmian}$$

$$s(2, 3) = 2 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5)$$

Aby algorytm zadziałał dobrze, problem musi spełniać warunki:

1. **Własność optymalnej podstruktury** – optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów.
2. **Własność wspólnych podproblemów** – sytuacja, gdy prosty algorytm rekurencyjny obliczałby rozwiązania tych samych podproblemów wielokrotnie. Liczba podproblemów powinna być „mała”.

W klasycznej metodzie programowania dynamicznego rozwiązanie jest budowane od dołu do góry – od podproblemów najmniejszych do największych. Jest też podejście, w którym rozwiązanie jest budowane od góry do dołu, podobnie jak w prostym algorytmie rekurencyjnym, ale z tą różnicą, że unikamy obliczania tych samych podproblemów wielokrotnie. To podejście nazywa się **spamiętywaniem** (*memoization*).

Problem mnożenia ciągu macierzy ze spamiętywaniem Algorytm oblicza tablice m i s jak poprzednio, ale w inny sposób. Dopóki podproblem nie jest rozwiązany, $m(i, j)$ przechowuje ∞ . Po rozwiązaniu podproblemu (i, j) umieszczamy w $m(i, j)$ odpowiednią wartość i od tej pory używamy jej bez obliczania.

Algorytm 9: Algorytm Lookup Chain

```

function LOOKUPCHAIN( $p, i, j$ )       $\triangleright p = (p_0, \dots, p_n)$  - ciąg wymiarów;  $A_i$  - macierz  $p_{i-1} \times p_i$ 
1:   if  $m(i, j) < \infty$  then
2:     return  $m(i, j), s(i, j)$ 
3:   if  $i = j$  then
4:      $m(i, j) \leftarrow 0$ 
5:   else
6:     for  $k \leftarrow i$  to  $j - 1$  do
7:        $q \leftarrow \text{LOOKUPCHAIN}(p, i, k) + \text{LOOKUPCHAIN}(p, k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
8:       if  $q < m(i, j)$  then
9:          $m(i, j) \leftarrow q$ 
10:         $s(i, j) \leftarrow k$ 
11:  return  $(m(i, j), s(i, j))$ 

```

Algorytm 10: Algorytm Memoized Matrix Chain

```

function MEMOIZEDMATRIXCHAIN( $p$ )
1:    $n \leftarrow \text{length}(p) - 1$ 
2:   for  $i \leftarrow 1$  to  $n$  do
3:     for  $j \leftarrow i$  to  $n$  do
4:        $m(i, j) \leftarrow \infty$ 
5:   return LOOKUPCHAIN( $p, 1, n$ )

```

Złożoność czasowa:

- Linie 1–4 w MEMOIZEDMATRIXCHAIN zajmują $\Theta(n^2)$.
- Każdy z $\mathcal{O}(n^2)$ elementów tablicy m jest modyfikowany raz, później odczytanie wartości $m(i, j)$ zajmuje czas stały.
- Modyfikacja $m(i, j)$ zajmuje $\mathcal{O}(n)$ czasu (pętla w liniach 5–9 w LOOKUPCHAIN).

Ostatecznie złożoność algorytmu wynosi $\mathcal{O}(n^3)$.

Problem najdłuższego wspólnego podciągu

Niech $X = (x_1, \dots, x_m)$ – ciąg.

$Z = (z_1, \dots, z_k)$ jest podciągiem X , jeśli istnieje rosnący ciąg indeksów $1 \leq i_1 < i_2 < \dots < i_k \leq m$ takich, że dla każdego $j \in \{1, \dots, k\}$, $z_j = x_{i_j}$.

Przykład: $X = (A, B, C, B, D, A, B)$, $Z = (B, C, D, B)$. Indeksy: (2, 3, 5, 7).

Problem: Dane są ciągi $X = (x_1, \dots, x_m)$ oraz $Y = (y_1, \dots, y_n)$. Należy znaleźć najdłuższy wspólny podciąg (LCS – *Longest Common Subsequence*) X i Y , czyli najdłuższy ciąg, który jest podciągiem zarówno X , jak i Y .

Wszystkich podciągów X jest 2^m , więc algorytm sprawdzający wszystkie podciągi X działa w czasie wykładniczym.

Niech $X_i = (x_1, \dots, x_i)$ dla $i \in \{1, \dots, m\}$ i dodatkowo $X_0 = ()$ (ciąg pusty). Analogicznie dla Y .

Lemat 3.1. Niech $X = (x_1, \dots, x_m)$, $Y = (y_1, \dots, y_n)$ i niech $Z = (z_1, \dots, z_k)$ będzie LCS X i Y .

1. Jeśli $x_m = y_n$, to $z_k = x_m = y_n$ i Z_{k-1} jest LCS X_{m-1} i Y_{n-1} .
2. Jeśli $x_m \neq y_n$ i $z_k \neq x_m$, to Z jest LCS X_{m-1} i Y .
3. Jeśli $x_m \neq y_n$ i $z_k \neq y_n$, to Z jest LCS X i Y_{n-1} .

Dowód.

1. Załóżmy, że $x_m = y_n$. Gdyby $z_k \neq x_m$, to moglibyśmy wydłużyć podciąg Z przez dodanie $z_{k+1} = x_m = y_n$, co daje sprzeczność, bo $(z_1, \dots, z_{k+1})$ jest wspólnym podciągiem X i Y dłuższym niż Z . Zatem $z_k = x_m = y_n$ i Z_{k-1} jest wspólnym podciągiem X_{m-1} i Y_{n-1} . Gdyby istniał wspólny podciąg W ciągów X_{m-1} i Y_{n-1} długości $> k - 1$, to dodając na koniec W element $x_m = y_n$ otrzymalibyśmy wspólny podciąg X i Y długości $> k - 1 + 1 = k$. Sprzeczność, bo długość Z wynosi k .
2. Załóżmy, że $x_m \neq y_n$ i $z_k \neq x_m$. Wtedy Z jest wspólnym podciągiem ciągów X_{m-1} i Y . Gdyby istniał wspólny podciąg W ciągów X_{m-1} i Y o długości $> k$, to W byłby wspólnym podciągiem ciągów X i Y dłuższym niż Z . Sprzeczność.
3. Symetrycznie do punktu 2.

□

Rozwiązanie dla problemu (X, Y) można skonstruować z rozwiązań podproblemów (X_{m-1}, Y_{n-1}) , (X_{m-1}, Y) i (X, Y_{n-1}) – **własność optymalnej podstruktury**. Podproblem (X_{m-1}, Y_{n-1}) jest zawarty w podproblemach (X_{m-1}, Y) i (X, Y_{n-1}) – **własność wspólnych podproblemów**.

Niech $c(i, j)$ oznacza długość LCS ciągów X_i i Y_j .

$$c(i, j) = \begin{cases} 0 & \text{jeśli } i = 0 \text{ lub } j = 0 \\ c(i-1, j-1) + 1 & \text{jeśli } i, j > 0 \text{ i } x_i = y_j \\ \max\{c(i-1, j), c(i, j-1)\} & \text{jeśli } i, j > 0 \text{ i } x_i \neq y_j \end{cases}$$

Będziemy też używać tablicy $b(i, j)$, aby znaleźć sam podciąg LCS. Tablice mają wymiary: $c[0 \dots m, 0 \dots n]$ oraz $b[1 \dots m, 1 \dots n]$. Wartości w tablicy kierunków to $b(i, j) \in \{\nwarrow, \uparrow, \leftarrow\}$.

Algorytm 11: Algorytm obliczający długość LCS

```

function LCS( $X, Y$ )
1:   $m \leftarrow \text{length}(X)$ 
2:   $n \leftarrow \text{length}(Y)$ 
3:  for  $i \leftarrow 0$  to  $m$  do
4:     $c(i, 0) \leftarrow 0$ 
5:  for  $j \leftarrow 0$  to  $n$  do
6:     $c(0, j) \leftarrow 0$ 
7:  for  $i \leftarrow 1$  to  $m$  do
8:    for  $j \leftarrow 1$  to  $n$  do
9:      if  $x_i = y_j$  then
10:         $c(i, j) \leftarrow c(i - 1, j - 1) + 1$ 
11:         $b(i, j) \leftarrow \text{,, } \nearrow \text{”}$ 
12:      else if  $c(i - 1, j) \geq c(i, j - 1)$  then
13:         $c(i, j) \leftarrow c(i - 1, j)$ 
14:         $b(i, j) \leftarrow \text{,, } \uparrow \text{”}$ 
15:      else
16:         $c(i, j) \leftarrow c(i, j - 1)$ 
17:         $b(i, j) \leftarrow \text{,, } \leftarrow \text{”}$ 
18:  return ( $c, b$ )

```

Przykład: $X = (A, B, C, B, D, A, B)$, $Y = (B, D, C, A, B, A)$.
 $m = 7$, $n = 6$.

	j	0	1	2	3	4	5	6
i		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	$\uparrow 0$	$\uparrow 0$	$\uparrow 0$	$\nearrow 1$	$\leftarrow 1$	$\nearrow 1$
2	B	0	$\nearrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\uparrow 1$	$\nearrow 2$	$\leftarrow 2$
3	C	0	$\uparrow 1$	$\uparrow 1$	$\nearrow 2$	$\leftarrow 2$	$\uparrow 2$	$\uparrow 2$
4	B	0	$\nearrow 1$	$\uparrow 1$	$\uparrow 2$	$\uparrow 2$	$\nearrow 3$	$\leftarrow 3$
5	D	0	$\uparrow 1$	$\nearrow 2$	$\uparrow 2$	$\uparrow 2$	$\uparrow 3$	$\uparrow 3$
6	A	0	$\uparrow 1$	$\uparrow 2$	$\uparrow 2$	$\nearrow 3$	$\uparrow 3$	$\nearrow 4$
7	B	0	$\nearrow 1$	$\uparrow 2$	$\uparrow 2$	$\uparrow 3$	$\nearrow 4$	$\uparrow 4$

Algorytm 12: Wypisywanie najdłuższego wspólnego podciągu

```

function PRINTLCS( $b, X, Y, i, j$ )
1:  if  $i = 0$  lub  $j = 0$  then
2:    return  $\varepsilon$ 
3:  if  $b(i, j) = \text{,, } \nearrow \text{”}$  then
4:    PRINTLCS( $b, X, Y, i - 1, j - 1$ )
5:    print  $x_i$ 
6:  else if  $b(i, j) = \text{,, } \uparrow \text{”}$  then
7:    PRINTLCS( $b, X, Y, i - 1, j$ )
8:  else
9:    PRINTLCS( $b, X, Y, i, j - 1$ )

```

▷ ciąg pusty

Żeby wypisać rozwiązanie, wywołujemy PRINTLCS(b, X, Y, m, n).
Dla powyższego przykładu otrzymamy: (B, C, B, A) .

Złożoność czasowa tego algorytmu wynosi $\Theta(m \cdot n)$.